

Algebraic Structures and Techniques

Algebraic rules of procedure were proclaimed as if they were divine revelations. . . .

—From *The History of Mathematics*
by David M. Burton

The word “algebra” comes from the word “al-jabr” in the title of the textbook *Hisâb al-jabr w'al-muqâbala*, which was written around 820 by the mathematician and astronomer al-Khowârizmî. The title translates roughly to “calculations by restoration and reduction,” where restoration—al-jabr—refers to adding or subtracting a number on both sides of an equation, and reduction refers to simplification. We should also note that the word “algorithm” has been traced back to al-Khowârizmî because people used his name—mispronounced—when referring to a method of calculating with Hindu numerals that was contained in another of his books.

Having studied high school algebra, most of us probably agree that algebra has something to do with equations and simplification. In high school algebra we simplified a lot. In fact, we were often given the one word command “simplify” in the exercises. So we tried to somehow manipulate a given expression into one that was simpler than the given one, although this direction was a bit vague, and there always seemed to be a question about what “simplify” meant. We also tried to describe word problems in terms of algebraic equations and then to apply our simplification methods to extract solutions. Everything we did dealt with numbers and expressions for numbers.

In this chapter we'll clarify and broaden the idea of an algebra. The chapter introduces the notions and notations of algebra with special emphasis on the techniques and applications of algebra in computer science.

chapter guide

- 10.1 (What Is an Algebra?)* introduces the idea of an algebra. We'll see that high school algebra is just one kind of algebra.
- 10.2 (Boolean Algebra)* introduces Boolean algebra together with some techniques to simplify Boolean expressions and digital circuits.
- 10.3 (Abstract Data Types as Algebras)* introduces the idea of an abstract data type as an algebra. As examples, we'll discuss some properties of the natural numbers, lists, strings, stacks, queues, binary trees, and priority queues.
- 10.4 (Computational Algebras)* introduces relational algebras and functional algebras. We'll see how these ideas are applied to databases and functional programming.
- 10.5 (Other Algebraic Ideas)* introduces a collection of algebraic ideas that are useful for computational problems. We'll introduce congruences and see how some results are applied in cryptology. We'll also introduce subalgebras and morphisms.

10.1 What Is an Algebra?

Before we say just what an algebra is, let's see how an algebra is used in the problem-solving process. An important part of problem solving is the process of transforming informal word problems into formal things like equations, expressions, or algorithms. Another important part of problem solving is the process of transforming these formal things into solutions by solving equations, simplifying expressions, or implementing algorithms. For example, in high school algebra we tried to describe certain word problems in terms of algebraic equations, and then we tried to solve the equations. An algebra should provide tools and techniques to help us describe informal problems in formal terms and to help us solve the resulting formal problems.

The Description Problem

How can we describe something to another person in such a way that the person understands exactly what we mean? One way is to use examples. But sometimes examples may not be enough for a proper understanding. It is often useful at some point to try to describe an object by describing some properties that it possesses. So we state the following general problem:

The Description Problem: Describe an object.

Whatever form a description takes, it should be communicated in a clear and concise manner so that examples or instances of the object can be easily checked for correctness. Try to describe one of the following things to a friend:

A car.

The left side of a person.

The number zero.

The concept of area.

Most likely, you'll notice that the description of an object often depends on the knowledge level of the audience.

We need some tools to help us describe properties of the things we are talking about, so we can check not only the correctness of examples, but also the correctness of the descriptions. Algebras provide us with natural notations that can help us give precise descriptions for many things, particularly those structures and ideas that are used in computer science.

High School Algebra

A natural example of an algebra that we all know and love is the algebra of numbers. We learned about it in school, and we probably had different ideas about what it was. First, we learned about arithmetic of the natural numbers $\mathbb{N}$, using the operation of addition. We came eventually to believe things like

$$7 + 12 = 19, \quad 3 + 5 = 5 + 3, \quad \text{and} \quad 4 + (6 + 2) = (4 + 6) + 2.$$

Soon we learned about multiplication, negative numbers, and the integers $\mathbb{Z}$. It seemed that certain numbers like 0 and 1 had special properties such as

$$14 + 0 = 14, \quad 1 \cdot 47 = 47, \quad \text{and} \quad 0 = 9 + (-9).$$

Somewhere along the line, we learned about division, the rational numbers $\mathbb{Q}$, and the fact that we could not divide by zero.

Then came the big leap. We learned to denote numbers by symbols like the letters x and y and by expressions like $x^2 + y$. We spent much time transforming one expression into another, such as $x^2 + 4x + 4 = (x + 2)(x + 2)$. All this had something to do with algebra, perhaps because that was the name of the class.

There are two main ingredients to the algebra that we studied in high school. The first is a set of numbers to work with, such as the real numbers $\mathbb{R}$. The second is a set of operations on the numbers, such as $-$ and $+$. We learned about the general properties of the operations, such as $x + y = y + x$ and $x + 0 = x$. And we learned to use these properties to simplify expressions and solve equations.

Now we are in position to discuss algebra from a more general point of view. We will see that high school algebra is just one of many different kinds of algebra.

10.1.1 Definition of an Algebra

An *algebra* is a structure consisting of one or more sets together with one or more operations on the sets. The sets are often called *carriers* of the algebra. This is a very general definition. If this is the definition of an algebra, how can it help us solve problems? As we will see, the utility of an algebra comes from knowing how to use the operations.

For example, high school algebra is an algebra with the single carrier $\mathbb{R}$, or maybe $\mathbb{Q}$. The operators of the algebra are $+$, $-$, $\cdot$, and $\div$. The constants 0 and 1 are also important to consider because they have special properties. Note that a constant can be thought of as a nullary operation (having arity zero). Many familiar properties hold among the operations, such as the fact that multiplication distributes over addition: $a \cdot (b + c) = a \cdot b + a \cdot c$; and the fact that we can cancel: If $a \neq 0$, then $a \cdot b = a \cdot c$ implies $b = c$.

Algebraic Expressions

An *algebraic expression* is a string of symbols used to represent an element in

a carrier of an algebra. For example, 3 , $8 \div x$, and $x^2 + y$ are algebraic expressions in high school algebra. But $x + y +$ is not an algebraic expression. The set of algebraic expressions is a language. The symbols in the alphabet are the operators and constants from the algebra together with variable names and grouping symbols, like parentheses and commas. The language of algebraic expressions over an algebra can be defined inductively as follows:

Algebraic Expressions

1. Constants and variables are algebraic expressions.
2. An operator applied to its arguments is an algebraic expression if the arguments are algebraic expressions.

For example, suppose x and y are variables and c is a constant. If g is a ternary operator, then the following five strings are algebraic expressions:

$$x, y, c, g(x, y, c), g(x, g(c, y, x), x).$$

Different algebraic expressions often mean the same thing. For example, the equation $2x = x + x$ makes sense to us because we look beyond the two strings $2x$ and $x + x$, which are not equal strings. Instead, we look at the possible values of the two expressions and conclude that they always have the same value, no matter what value x has. Two algebraic expressions are *equivalent* if they always evaluate to the same element in a carrier of the algebra. So the expressions $2x$ and $x + x$ are equivalent in high school algebra. We can make the idea of equivalence precise by giving an inductive definition. Assume that C is a carrier of an algebra.

Equivalent Algebraic Expressions

1. Any element in C is equivalent to itself.
2. Suppose E and E' are two algebraic expressions and x is a variable such that $E(x/b)$ and $E'(x/b)$ are equivalent for all elements b in C . Then E is equivalent to E' .

For example, the two expressions $(x + 2)^2$ and $x^2 + 4x + 4$ are equivalent in high school algebra. But $x + y$ is not equivalent to $5x$ because we can let $x = 1$ and $y = 2$, which makes $x + y = 3$ and $5x = 5$.

Describing an Algebra

The set of operators in an algebra is called the *signature* of the algebra. When describing an algebra, we need to decide which operators to put in the signature. For example, we may wish to list only the primitive operators (the constructors) that are used to build all other operators. On the other hand, we might want to list all the operators that we know about.

Let's look at a convenient way to denote an algebra. We'll list the carrier or carriers first, followed by a semicolon. The operators in the signature are listed next. For example, this notation is used to denote the following algebras.

$$\begin{array}{ll} \langle \mathbb{N}; \text{succ}, 0 \rangle & \langle \mathbb{N}; +, \cdot, 0, 1 \rangle \\ \langle \mathbb{N}; +, 0 \rangle & \langle \mathbb{Z}; +, \cdot, -, 0, 1 \rangle \\ \langle \mathbb{N}; \cdot, 1 \rangle & \langle \mathbb{Q}; +, \cdot, -, \div, 0, 1 \rangle \\ \langle \mathbb{N}; \text{succ}, +, 0 \rangle & \langle \mathbb{R}; +, \cdot, -, \div, 0, 1 \rangle \end{array}$$

The constants 0 and 1 are listed as operations to emphasize the fact that they have special properties, such as $x + 0 = x$ and $x \cdot 1 = x$.

It may also be convenient to use a picture to describe an algebra. The diagram in [Figure 10.1](#) represents the algebra $\langle \mathbb{N}; +, 0 \rangle$.

The circle represents the carrier $\mathbb{N}$, of natural numbers. The two arrows coming out of $\mathbb{N}$ represent two arguments to the $+$ operator. The arrow from $+$ to $\mathbb{N}$ indicates that the result of $+$ is an element of $\mathbb{N}$. The fact that there are no arrows pointing at 0 means that 0 is a constant (an operator with no arguments), and the arrow from 0 to $\mathbb{N}$ means that 0 is an element of $\mathbb{N}$.

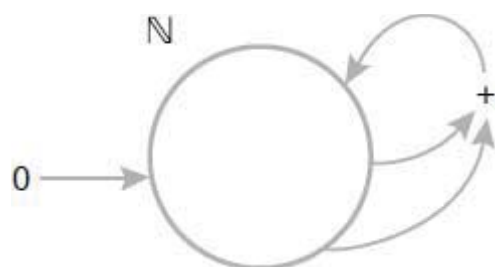


Figure 10.1 An algebra.

10.1.2 Concrete Versus Abstract

An algebra is called *concrete* if its carriers are specific sets of elements so that its operators are defined by rules applied to the carrier elements. High school algebra is a concrete algebra. In fact, all the examples that we have seen so far are concrete algebras.

An algebra is called *abstract* if it is not concrete. In other words, its carriers don't have any specific set interpretation. Thus its operators cannot be defined in terms of rules applied to the carrier elements because we don't have a description of them. Therefore the general properties of the operators in an abstract algebra must be given by axioms. An abstract algebra is a powerful description tool because it represents all the concrete algebras that satisfy its axioms.

So when we talk about an abstract algebra, we are really talking about all possible examples of the algebra. Is this a useful activity? Sure. Many times we are overwhelmed with important concepts, but we aren't given any tools to make sense of them. Abstraction can help to classify things and thus make sense of things that act in similar ways.

If an algebra is abstract, then we must be more explicit when trying to describe it. For example, suppose we write down the following algebra:

$$\langle S; s, a \rangle.$$

All we know at this point is that S is a carrier and there are two operators s and a . We don't even know the arity of s or a .

Suppose we're told that a is a constant of S and s is a unary operator on S . Now we know something, but not very much, about the algebra. We can use the operators s and a to construct the following algebraic expressions for elements of S :

$$a, s(a), s(s(a)), \dots, s^n(a), \dots$$

This is the most we can say about the elements of S . There might be other elements in S , but we have no way of knowing it. The elements of S that we know about can be represented by all possible algebraic expressions made up

from the operator symbols in the signature together with left and right parentheses.

example 10.1 Induction Algebra

An algebra $\langle S; s, a \rangle$ is called an *induction algebra* if s is a unary operator on S and a is a constant of S such that

$$S = \{a, s(a), s(s(a)), \dots, s^n(a), \dots\}.$$

The word “induction” is used because of the natural ordering on the carrier that can be used in inductive proofs.

The algebra $\mathbb{N}; \text{succ}, 0$ is a concrete example of an induction algebra, where $\text{succ}(x) = x + 1$. The algebraic expressions for the elements are

$$0, \text{succ}(0), \text{succ}(\text{succ}(0)), \dots, \text{succ}^n(0), \dots,$$

where $\text{succ}(0) = 1$, $\text{succ}(\text{succ}(0)) = 2$, and so on. So every natural number is represented by one of the algebraic expressions.

There are many concrete examples of induction algebras. For example, let $A = \{2, 1, 0, -1, -2, -3, \dots\}$. Then the algebra $A; \text{pred}, 2$ is an induction algebra, where $\text{pred}(x) = x - 1$. The expressions for the elements are

$$2, \text{pred}(2), \text{pred}(\text{pred}(2)), \dots, \text{pred}^n(2), \dots,$$

where we have $\text{pred}(2) = 1$, $\text{pred}(\text{pred}(2)) = 0$, and so on. So every number in A is represented by one of the algebraic expressions.

end example

Axioms for Abstractions

Interesting things can happen when we add axioms to an abstract algebra. For example, the algebra $S; s, a$ changes its character when we add the single axiom $s^6(x) = x$ for all $x \in S$. In this case we can say that the algebraic expressions define a finite set of elements, which can be represented by the following six expressions:

$$a, s(a), s^2(a), s^3(a), s^4(a), s^5(a).$$

A complete definition of an abstract algebra can be given by listing the carriers, operations, and axioms. For example, the abstract algebra that we've just been discussing can be defined as follows.

Carrier: S

Operations: $a \in S$
 $s : S \rightarrow S$

Axiom: $s^6(x) = x$.

We'll always assume that the variable x is universally quantified over S .

example 10.2 A Finite Algebra

The algebra $\langle \mathbb{N}_6; \text{succ}_6, 0 \rangle$, where $\text{succ}_6(x) = (x + 1) \bmod 6$, is a concrete example of the abstract algebra $\langle S; s, a \rangle$ with axiom $s^6(x) = x$. To see this, observe that the algebraic expressions for the carrier elements are

$$0, \text{succ}_6(0), \text{succ}_6(\text{succ}_6(0)), \dots, \text{succ}_6^n(x), \dots$$

But we have $\text{succ}_6(x) = (x + 1) \bmod 6$. So the preceding algebraic expressions evaluate to an infinite repetition of six numbers in $\mathbb{N}_6$.

$$0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, \dots$$

In other words, we have $\text{succ}_6^6(x) = x$ for all $x \in \mathbb{N}_6$, which has the same form as the axiom in the abstract algebra.

end example

10.1.3 Working in Algebras

The goal of the following paragraphs is to get familiar with some elementary properties of algebraic operations. We'll see more examples of algebras and we'll observe whether they have any of the properties we've discussed.

Properties of the Operations

Let's look at some fundamental properties that may be associated with a binary operation. If $\circ$ is a binary operator on a set C , then an element $z \in C$ is called a *zero* for $\circ$ if the following condition holds:

$$z \circ x = x \circ z = z \quad \text{for all } x \in C.$$

For example, the number 0 is a zero for the multiply operation over the real numbers because $0 \cdot x = x \cdot 0 = 0$ for all real numbers x .

Continuing with the same binary operator $\circ$ and carrier C , we call an element $u \in C$ an *identity*, or *unit*, for $\circ$ if the following condition holds:

$$u \circ x = x \circ u = x \quad \text{for all } x \in C.$$

For example, the number 1 is an identity for the multiply operation over the real numbers because $1 \cdot x = x \cdot 1 = x$ for all numbers x . Similarly, the number 0 is an identity for the addition operation over real numbers because $0 + x = x + 0 = x$ for all numbers x .

Suppose u is an identity element for $\circ$, and $x \in C$. An element y in C is called an *inverse* of x if the following equation holds:

$$x \circ y = y \circ x = u.$$

For example, in the algebra $\langle \mathbb{Q}; \cdot, 1 \rangle$ the number 1 is an identity element. We also know that if $x \neq 0$, then

$$x \cdot \frac{1}{x} = \frac{1}{x} \cdot x = 1.$$

In other words, all nonzero rational numbers have inverses.

Each of the following examples presents an algebra together with some observations about its operators.

example 10.3 Algebra of Sets

Let S be a set. Then the power set of S is the carrier for an algebra described as follows:

$$\langle \text{power}(S); \cup, \cap, \emptyset, S \rangle.$$

Notice that if $A \in \text{power}(S)$, then $A \cup \emptyset = A$, and $A \cap S = A$. So $\emptyset$ is an identity for $\cup$, and S is an identity for $\cap$. Similarly, $A \cap \emptyset = \emptyset$, and $A \cup S = S$. Thus $\emptyset$ is a zero for $\cap$, and S is a zero for $\cup$. This algebra has many well-known properties. For example, $A \cup A = A$ and $A \cap A = A$ for any $A \in \text{power}(S)$. We also know that $\cap$ and $\cup$ are commutative and associative and that they distribute over each other.

end example

example 10.4 A Finite Algebra

Let $\mathbb{N}_n$ denote the set $\{0, 1, \dots, n-1\}$, and let “max” be the function that returns the maximum of its two arguments. Consider the following algebra with carrier $\mathbb{N}_n$:

$$\langle \mathbb{N}_n; \max, 0, n-1 \rangle.$$

Notice that max is commutative and associative. Notice also that for any $x \in \mathbb{N}_n$ it follows that $\max(x, 0) = \max(0, x) = x$. So 0 is an identity for max. It’s also easy to see that for any $x \in \mathbb{N}_n$,

$$\max(x, n-1) = \max(n-1, x) = n-1.$$

So $n-1$ is a zero for the operator max.

end example

example 10.5 An Algebra of Functions

Let S be a set, and let F be the set of all functions of type $S \rightarrow S$. If we let $\circ$ denote the operation of composition of functions, then F is the carrier of an algebra $F; \circ, \text{id}$. The function “id” denotes the identity function. In other words, we have the equation $\text{id} \circ f = f \circ \text{id} = f$ for all functions f in F . Therefore, id is an identity for $\circ$.

Notice that we used the equality symbol “=” in the previous examples without explicitly defining it as a relation. The first example uses equality of sets, the second uses equality of numbers, and the third uses equality of functions. In our discussions we will usually assume an implicit equality theory on each carrier of an algebra. But, as we have said before, equality relations are operations that may need to be implemented when needed as part of a programming activity.

$\circ$	a	b	c	d
a	a	b	c	d
b	b	c	d	a
c	c	d	a	b
d	d	a	b	c

Figure 10.2 Binary operation table.

Operation Tables

Any binary operation on a finite set can be represented by a table, called an *operation table*. For example, if $\circ$ is a binary operation on the set $\{a, b, c, d\}$, then the operation table for $\circ$ might look like Figure 10.2, where the elements of the set are used as row labels and column labels.

If x is a row label and y is a column label, then the element in the table at row x and column y represents the element $x \circ y$. For example, we have $c \circ d = b$.

We can often find out many things about a binary operation by observing its operation table. For example, notice in Figure 10.2 that the row labeled a and the column labeled a are copies of the row label and column label sequence $a \ b \ c \ d$. This tells us that a is an identity for $\circ$. It's also easy to see that $\circ$ is commutative and that each element has an inverse. Does $\circ$ have a zero? It's easy to see that the answer is no. Is $\circ$ associative? The answer is yes, but it's not very easy to check. We'll leave these problems as exercises.

It's also easy to see that there cannot be more than one identity for a binary

operation. Can you see why from Figure 10.2? We'll prove the following general fact about identities for any binary operation.

Uniqueness of an Identity

(10.1)

Any binary operation has at most one identity.

Proof: Let $\circ$ be a binary operation on a set S . To show that $\circ$ has at most one identity, we'll assume that u and e are identities for $\circ$. Then we'll show that $u = e$. Remember, since u and e are identities, we know that $u \circ x = x \circ u = x$ and $e \circ x = x \circ e = x$ for all x in S . Thus we have the following equality:

$$\begin{aligned} e &= e \circ u && (u \text{ is an identity for } \circ) \\ &= u && (e \text{ is an identity for } \circ). \quad \text{QED.} \end{aligned}$$

Algebras with One Binary Operation

Some algebras are used so frequently that they have been given names. For example, any algebra of the form $\langle A; \circ \rangle$, where $\circ$ is a binary operation, is called a *groupoid*. If we know that the binary operation is associative, then the algebra is called a *semigroup*. If we know that the binary operation is associative and also has an identity, then the algebra is called a *monoid*. If we know that the binary operation is associative, has an identity, and each element has an inverse, then the algebra is called a *group*. So these words are used to denote certain properties of the binary operation. Here's a synopsis.

- Groupoid: $\circ$ is a binary operation.
- Semigroup: $\circ$ is an associative binary operation.
- Monoid: $\circ$ is an associative binary operation with an identity.
- Group: $\circ$ is an associative binary operation with an identity and every element has an inverse.

It's clear from the listing of properties that a group is a monoid, which is a semigroup, which is a groupoid. But things don't go the other way. For

example, the algebra in Example 10.5 is a monoid but not a group, since not every function has an inverse.

We can have some fun with these names. For example, we can describe a group as a monoid with inverses, and we can describe a monoid as a semigroup with identity. When an algebra contains an operation that satisfies some special property beyond the axioms of the algebra, we often modify the name of the algebra with the name of the property. For example, the algebra $\langle \mathbb{Z}; +, 0 \rangle$ is a group. But we know that the operation $+$ is commutative. Therefore, we can call the algebra a “commutative” group.

Now let’s discuss a few elementary results. To get our feet wet, we’ll prove the following simple property that holds in any monoid.

Uniqueness of Inverses

(10.2)

In a monoid, if an element has an inverse, the inverse is unique.

Proof: Let $\langle M; \circ, u \rangle$ be a monoid. We will show that if an element x in M has an inverse, then the inverse is unique. In other words, if y and z are both inverses of x , then $y = z$. We can prove this result as follows:

$$\begin{aligned}
 y &= y \circ u && (u \text{ is the identity for } \circ) \\
 &= y \circ (x \circ z) && (z \text{ is an inverse of } x) \\
 &= (y \circ x) \circ z && (\circ \text{ is associative}) \\
 &= u \circ z && (y \text{ is an inverse of } x) \\
 &= z && (u \text{ is the identity for } \circ). \quad \text{QED.}
 \end{aligned}$$

If we have a group, then we know that every element has an inverse. Thus we can conclude from (10.2) that every element x in a group has a unique inverse, which is usually denoted by writing the symbol

$$x^{-1}.$$

example 10.6 Working with Groups

We can use the elementary properties of a group to obtain other properties. For example, let $\langle G; \circ, e \rangle$ be a group. This means that we know that $\circ$ is associative, e is an identity, and every element of G has an inverse. We'll prove two properties as examples. The first property is stated as follows:

$$\text{If } x \circ x = x \text{ holds for some element } x \in G, \text{ then } x = e. \quad (10.3)$$

$$\begin{aligned} \text{Proof: } x &= x \circ e && (e \text{ is an identity for } \circ) \\ &= x \circ (x \circ x^{-1}) && (x^{-1} \text{ is the inverse of } x) \\ &= (x \circ x) \circ x^{-1} && (\circ \text{ is associative}) \\ &= x \circ x^{-1} && (x \circ x = x \text{ is the hypothesis}) \\ &= e && (x^{-1} \text{ is the inverse of } x). \text{ QED.} \end{aligned}$$

A second property of groups is cancellation on the left. This property can be stated as follows:

$$\text{If } x \circ y = x \circ z \text{ then } y = z. \quad (10.4)$$

$$\begin{aligned} \text{Proof: } y &= e \circ y && (e \text{ is an identity}) \\ &= (x^{-1} \circ x) \circ y && (x^{-1} \text{ is the inverse of } x) \\ &= x^{-1} \circ (x \circ y) && (\circ \text{ is associative}) \\ &= x^{-1} \circ (x \circ z) && (\text{hypothesis}) \\ &= (x^{-1} \circ x) \circ z && (\circ \text{ is associative}) \\ &= e \circ z && (x^{-1} \text{ is the inverse of } x) \\ &= z && (e \text{ is an identity}). \text{ QED.} \end{aligned}$$

end example

Algebras with Several Operations

A natural example of an algebra with two binary operations is the integers

together with the usual operations of addition and multiplication. We can denote this algebra by the structure $\langle \mathbb{Z}; +, \cdot, 0, 1 \rangle$. This algebra is a concrete example of an algebra called a ring, which we'll now define. A *ring* is an algebra with the structure

$$A; +, \cdot, 0, 1,$$

$+_5$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

$\cdot_5$	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Figure 10.3 Mod 5 addition and multiplication tables.

where $A; +, 0$ is a commutative group, $A; \cdot, 1$ is a monoid, and the operation $\cdot$ distributes over $+$ from the left and the right. This means that

$$a \cdot (b + c) = a \cdot b + a \cdot c \text{ and } (b + c) \cdot a = b \cdot a + c \cdot a.$$

Check to see that $\mathbb{Z}; +, \cdot, 0, 1$ is indeed a ring.

If $A; +, \cdot, 0, 1$ is a ring with the additional property that $A - \{0\}; \cdot, 1$ is a commutative group, then it's called a *field*. The ring $\mathbb{Z}; +, \cdot, 0, 1$ is not a field because, for example, 3 does not have an inverse for multiplication. On the other hand, if we replace $\mathbb{Z}$ by $\mathbb{Q}$, the rational numbers, then $\mathbb{Q}; +, \cdot, 0, 1$ is a field. For example, 3 has inverse $1/3$ in $\mathbb{Q} - \{0\}$.

For another example of a field, let $\mathbb{N}_5 = \{0, 1, 2, 3, 4\}$ and let $+_5$ and $\cdot_5$ be addition mod 5 and multiplication mod 5, respectively. Then $\mathbb{N}_5; +_5, \cdot_5, 0, 1$ is a field. Figure 10.3 shows the operation tables for $+_5$ and $\cdot_5$. We'll leave the verification of the field properties as an exercise.

The next examples show some algebras that might be familiar to you.

example 10.7 Polynomial Algebras

Let $\mathbb{R}[x]$ denote the set of all polynomials over x with real numbers as coefficients. It's a natural process to add and multiply two polynomials. So we have an algebra $\langle \mathbb{R}[x]; +, \cdot, 0, 1 \rangle$, where $+$ and $\cdot$ represent addition and multiplication of polynomials and 0 and 1 represent themselves. This algebra is a ring. Why isn't it a field?

end example

example 10.8 Matrix Algebras

Suppose we let $M_n(\mathbb{R})$ denote the set of all n by n matrices with elements in $\mathbb{R}$. We can add two matrices A and B by letting $A_{ij} + B_{ij}$ be the element in the i th row and j th column of the sum. We can multiply A and B by letting $\sum_{k=1}^n A_{ik} B_{kj}$ be the element in the i th row and j th column of the product. Thus we have an algebra $\langle M_n(\mathbb{R}); +, \cdot, 0, 1 \rangle$, where $+$ and $\cdot$ represent matrix addition and multiplication, 0 represents the matrix with all entries zero, and 1 represents the matrix with 1's along the main diagonal and 0's elsewhere. This algebra is a ring. Why isn't it a field?

end example

example 10.9 Vector Algebras

The algebra of n -dimensional vectors, with real numbers as components, can be described by listing two carriers $\mathbb{R}$ and $\mathbb{R}^n$. We can multiply a vector $(x_1, \dots, x_n) \in \mathbb{R}^n$ by number $b \in \mathbb{R}$ to obtain a new vector by multiplying each component of the vector by b , obtaining

$$(bx_1, \dots, bx_n).$$

If we let $\cdot$ denote this operation, then we have

$$b \cdot (x_1, \dots, x_n) = (bx_1, \dots, bx_n).$$

We can add vectors by adding corresponding components. For example,

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n).$$

Thus we have an algebra $\langle \mathbb{R}, \mathbb{R}^n; \cdot, + \rangle$ of n -dimensional vectors. Notice that the algebra has two carriers, $\mathbb{R}$ and $\mathbb{R}^n$. This is because they are both necessary to define the $\cdot$ operation, which has type $\mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$.

end example

example 10.10 Power Series Algebras

If we extend polynomials over x to allow infinitely many terms, then we obtain what are called *power series* (we also know them as generating functions). Letting $\mathbb{R}[[x]]$ denote the set of power series with real numbers as coefficients, we obtain the algebra $\langle \mathbb{R}[[x]]; +, \cdot, 0, 1 \rangle$, where $+$ and $\cdot$ represent addition and multiplication of power series and 0 and 1 represent themselves. This algebra is a ring. Why isn't it a field?

end example

Exercises

Algebraic Properties

1. Let m and n be two integers with $m < n$. Let $A = \{m, m + 1, \dots, n\}$, and let “min” be the function that returns the smaller of its two arguments. Does min have a zero? Identity? Inverses? If so, describe them.
2. Let $A = \{\text{True}, \text{False}\}$. For each of the following binary operations on A , answer the three questions: Does the operation have a zero? Does the operation have an identity? What about inverses?
 - a. Conditional, $\rightarrow$.
 - b. Conjunction, $\wedge$.
 - c. Disjunction, $\vee$.
3. Given the algebra $\langle S; f, a \rangle$, where f is a unary operation and a is a constant

of S and $f^5(x) = f^3(x)$ for all $x \in S$, find a finite set of algebraic expressions that will represent the distinct elements of S .

4. Given a binary operation on a finite set in table form, for each of the following parts, describe an easy way to detect whether the binary operation has the listed property.
 - a. There is a zero.
 - b. The operation is commutative.
 - c. Inverses exist for each element of the set (assume there is an identity).
5. Let $A = \{a, b, c, d\}$, and let $\circ$ be a binary operation on A . For each of the following problems, write down a table for $\circ$ that satisfies the given properties.
 - a. a is an identity, but no other element of A has an inverse.
 - b. a is an identity, and every element of A has an inverse.
 - c. a is a zero, and $\circ$ is not associative.
 - d. a is an identity, and exactly two elements have inverses.
 - e. a is an identity, and $\circ$ is commutative but not associative.
6. Let $A = \{a, b\}$. For each of the following problems, find an operation table satisfying the given condition for a binary operation $\circ$ on A .
 - a. $\langle A; \circ \rangle$ is a group.
 - b. $A; \circ$ is a monoid but not a group.
 - c. $A; \circ$ is a semigroup but not a monoid.
 - d. $A; \circ$ is a groupoid but not a semigroup.
7. Write an algorithm to check a binary operation table for associativity.

Challenges

8. Given the algebra $S; f, g, a$, where f and g are unary operations and a is a constant of S , suppose that $f(f(x)) = g(x)$ and $g(g(x)) = x$ for all $x \in S$.
 - a. Show that $f(g(x)) = g(f(x))$ for all $x \in S$.
 - b. Show that $f(f(f(f(x)))) = x$ for all $x \in S$.
 - c. Find a finite set of algebraic expressions to represent the distinct

elements of S .

9. Prove each of the following facts about a group $\langle G; \circ, e \rangle$.

a. Cancellation on the right: If $y \circ x = z \circ x$, then $y = z$.

b. The inverse of $x \circ y$ is $y^{-1} \circ x^{-1}$. In other words, $(x \circ y)^{-1} = y^{-1} \circ x^{-1}$.

10. Let $\mathbb{N}_5 = \{0, 1, 2, 3, 4\}$, and let $+_5$ and $\cdot_5$ be the two operations of addition mod 5 and multiplication mod 5, respectively. Show that $\langle \mathbb{N}_5; +_5, \cdot_5, 0, 1 \rangle$ is a field.

10.2 Boolean Algebra

Do the techniques of set theory and the techniques of logic have anything in common? Let's do an example to see that the answer is yes. When working with sets, we know that the following equation holds for all sets A , B , and C :

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C).$$

When working with propositions, we know that the following equivalence holds for all propositions A , B , and C :

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C).$$

Certainly these two examples have a similar pattern. As we'll see shortly, sets and logic have a lot in common. They can both be described as concrete examples of a Boolean algebra. The name "Boolean" comes from the mathematician George Boole (1815–1864), who studied relationships between set theory and logic. Let's get to the definition.

Definition of Boolean Algebra

A *Boolean algebra* is an algebra with the structure $\langle B; +, \cdot, -, 0, 1 \rangle$, where the following properties hold.

Defining Properties of a Boolean Algebra

1. $\langle B; +, 0 \rangle$ and $\langle B; \cdot, 1 \rangle$ are commutative monoids. In other words, the following properties hold for all $x, y, z \in B$:

$$(x + y) + z = x + (y + z), \quad (x \cdot y) \cdot z = x \cdot (y \cdot z),$$

$$x + y = y + x, \quad x \cdot y = y \cdot x,$$

$$x + 0 = x, \quad x \cdot 1 = x.$$

2. The operations $+$ and $\cdot$ distribute over each other. In other words, the following properties hold for all $x, y, z \in B$:

$$x \cdot (y + z) = (x \cdot y) + (x \cdot z) \text{ and } x + (y \cdot z) = (x + y) \cdot (x + z).$$

3. $x + \overline{x} = 1$ and $x \cdot \overline{x} = 0$ for all elements $x \in B$. The element $\overline{x}$ is called the *complement* of x or the *negation* of x .

We often drop the dot and write xy in place of $x \cdot y$. We'll also reduce the need for parentheses by agreeing to the following precedence hierarchy:

$\overline{}$ highest (do it first),

$\cdot$

$+$ lowest (do it last).

For example, the expression $x + y\overline{z}$ means the same thing as $(x + (y(\overline{z})))$.

example 10.11 Sets

Suppose $B = \text{power}(S)$ for some set S . Then B is the carrier of a Boolean algebra if we let union and intersection act as the operations $+$ and $\cdot$, let X' be the complement of X , let $\emptyset$ act as 0, and let S act as 1. For example, the two properties in Part 3 of the definition are represented by the following equations, where X is any subset of S :

$$X \cup X' = S \quad \text{and} \quad X \cap X' = \emptyset.$$

end example

example 10.12 Logic

Suppose we let B be the set of all propositional wffs of the propositional calculus. Then B is the carrier of a Boolean algebra if we let disjunction and conjunction act as the operations $+$ and $\cdot$, let $\neg X$ be the complement of X , let False act as 0, let True act as 1, and let logical equivalence act as equality. For example, the two properties in Part 3 of the definition are represented by the following equivalences, where X is any proposition:

$$X \vee \neg X \equiv \text{True} \quad \text{and} \quad X \wedge \neg X \equiv \text{False}.$$

We can also obtain a very simple Boolean algebra by using just the carrier $\{\text{False}, \text{True}\}$ together with the operations $\vee$, $\wedge$, and $\neg$.

end example

example 10.13 Divisors

Let n be a product of distinct prime numbers. For example, n could be 30 because $30 = 2 \cdot 3 \cdot 5$, but n cannot be 12 because $12 = 2 \cdot 2 \cdot 3$, which is not a product of distinct primes. Let B_n be the set of positive divisors of n . Then B_n is the carrier of a Boolean algebra if we let “least common multiple” and “greatest common divisor” be the operations $+$ and $\cdot$, respectively. Let n/x be the complement of x , let 1 act as the zero, and let n act as the one.

With these definitions, all the properties of a Boolean algebra are satisfied. For example, the two properties in Part 3 of the definition are represented by the following equations, where $x \in B_n$:

$$\text{lcm}(x, n/x) = n \quad \text{and} \quad \text{gcd}(x, n/x) = 1.$$

For an example, let $n = 10 = 2 \cdot 5$. Then $B_{10} = \{1, 2, 5, 10\}$, 1 is the zero, and 10 is the one. Thus, for example, the complement of 2 is 5, $\text{lcm}(2, 5) = 10$ (the one), and $\text{gcd}(2, 5) = 1$ (the zero).

Notice what happens if we let $n = 12$. We get $B_{12} = \{1, 2, 3, 4, 6, 12\}$. The reason B_{12} does not yield a Boolean algebra with our definition is because 2 and its complement 6 don't satisfy the properties in Part 3 of the definition. Notice that $\text{lcm}(2, 6) = 6$, which is not the one, and $\text{gcd}(2, 6) = 2$, which is not the zero.

end example

10.2.1 Simplifying Boolean Expressions

A fundamental problem of Boolean algebra, with applications to such areas as logic design and theorem-proving systems, is to simplify Boolean expressions so that they contain a small number of operations. Let's see how the axioms of Boolean algebra can help us obtain some useful simplification properties.

For example, in the Boolean algebra of propositions we have $P \wedge P \equiv P$ for any proposition P . Similarly, in the Boolean algebra of sets we have $S \cap S = S$ for any set S . Can we generalize these properties to all Boolean algebras? In other words, can we say $xx = x$ for every element x in the carrier of a Boolean algebra? The answer is yes. Let's prove it with equational reasoning. Be sure you can provide a reason for each step of the following proof:

$$x = x \cdot 1 = x \cdot (x + \overline{x}) = x \cdot x + x \cdot \overline{x} = x \cdot x + 0 = x \cdot x.$$

A related statement is $x + x = x$ for all elements x . Can you provide the proof? We'll state these two properties for the record.

Idempotent Properties

(10.5)

$$x \cdot x = x \quad \text{and} \quad x + x = x.$$

A nice property of Boolean algebras is that results come in pairs. This is because the axioms come in pairs. In other words, $\langle B; +, 0 \rangle$ and $\langle B; \cdot, 1 \rangle$ are both commutative monoids; $+$ and $\cdot$ distribute over each other; and $x + \overline{x} = 1$ and $x \cdot \overline{x} = 0$ for all elements $x \in B$. The *duality principle* states that

whenever a result A is true for a Boolean algebra, then a dual result A' is also true, where A' is obtained from the A by simultaneously replacing all occurrences of $\cdot$ by $+$, all occurrences of $+$ by $\cdot$, all occurrences of 1 by 0, and all occurrences of 0 by 1. A proof for the result A' can be obtained by making these same changes in the proof of A .

There are lots of properties that we can discover. For example, if S is a set, then $\emptyset \cap A = \emptyset$ for any subset A of S . This is an instance of a general property that holds for any Boolean algebra: $0 \cdot x = 0$ for every element x . This follows readily from (10.5) as follows:

$$0 \cdot x = (\overline{x} \cdot x) \cdot x = \overline{x} \cdot (x \cdot x) = \overline{x} \cdot x = 0.$$

Again, there is a dual result: $1 + x = 1$ for every x . Can you prove this result? We'll also state these two properties for the record:

Zero and One Properties

(10.6)

$$0 \cdot x = 0 \quad \text{and} \quad 1 + x = 1.$$

Let's do an example to see how we can put our new knowledge to use in simplifying a Boolean expression. Suppose the function f is defined over a Boolean algebra by

$$f(x, y, z) = x + yz + z\overline{x}y + \overline{y}xz.$$

To evaluate f , we need to perform three $+$ operations, five $\cdot$ operations, and two $-$ operations. Can we do any better? Sure we can. We can simplify the expression for $f(x, y, z)$ as follows—make sure you can state a reason for each line:

$$\begin{aligned}
 f(x, y, z) &= x + yz + z\bar{x}y + \bar{y}xz \\
 &= x + yz(1 + \bar{x}) + \bar{y}xz \\
 &= x + yz1 + \bar{y}xz \\
 &= x(1 + \bar{y}z) + yz \\
 &= x1 + yz \\
 &= x + yz.
 \end{aligned}$$

So f can be evaluated with only one $+$ operation and one $\cdot$ operation.

To simplify Boolean expressions, it's important to have a good knowledge of Boolean algebra together with some luck and ingenuity. We'll give a few more general properties that are very useful simplification tools. The following properties can be used to simplify an expression by reducing the number of operations by two. We'll leave the proofs as exercises.

Absorption Laws

(10.7)

$$\begin{aligned}
 x + xy &= x & \text{and} & & x(x + y) &= x. \\
 x + \bar{x}y &= x + y & \text{and} & & x(\bar{x} + y) &= xy.
 \end{aligned}$$

In a Boolean algebra, complements are unique in the following sense: If an element acts like a complement of some element, then it is in fact the only complement of the element. Using symbols, we can state the result as follows:

Uniqueness of the Complement

(10.8)

$$\text{If } x + y = 1 \text{ and } xy = 0, \text{ then } y = \bar{x}.$$

Proof: To prove this statement, we write the following equations:

$$\begin{aligned}
y &= y1 \\
&= y(x + \bar{x}) \\
&= yx + y\bar{x} \\
&= 0 + y\bar{x} && (\text{since } xy = 0) \\
&= x\bar{x} + y\bar{x} \\
&= (x + y)\bar{x} \\
&= 1\bar{x} && (\text{since } x + y = 1) \\
&= \bar{x} && \text{QED.}
\end{aligned}$$

Complements are quite useful in Boolean algebra. As a consequence of the uniqueness of complements (10.8), we have the following property:

Involution Law

(10.9)

$$\overline{\overline{x}} = x.$$

Proof: Notice that $\overline{x} + x = 1$ and $\overline{\overline{x}}x = 0$. Therefore, x acts like the complement of $\overline{x}$. Thus x is indeed equal to the complement of $\overline{x}$. That is, $x = \overline{\overline{x}}$. QED.

Recall from the propositional calculus that we have the following logical equivalence $\neg(p \wedge q) \equiv \neg p \vee \neg q$. This is an example of one of De Morgan's laws, which have the following forms in Boolean algebra.

De Morgan's Laws

(10.10)

$$\overline{x + y} = \overline{x}\overline{y} \quad \text{and} \quad \overline{xy} = \overline{x} + \overline{y}.$$

Proof: We'll prove the first of the two laws and leave the second as an exercise. We'll show that $\overline{x}\overline{y}$ acts like the complement of $x + y$. Then we'll

use (10.8) to conclude the result. First, we'll show that $(x + y) + \bar{x}\bar{y} = 1$ as follows:

$$\begin{aligned}(x + y) + \bar{x}\bar{y} &= (x + y + \bar{x})(x + y + \bar{y}) \\ &= (x + \bar{x} + y)(x + y + \bar{y}) \\ &= (1 + y)(x + 1) \\ &= 1 \cdot 1 \\ &= 1.\end{aligned}$$

Next we'll show that $(x + y)\bar{x}\bar{y} = 0$ as follows:

$$\begin{aligned}(x + y)\bar{x}\bar{y} &= x\bar{x}\bar{y} + y\bar{x}\bar{y} \\ &= x\bar{x}\bar{y} + \bar{x}y\bar{y} \\ &= 0\bar{y} + \bar{x}0 \\ &= 0 + 0 \\ &= 0.\end{aligned}$$

Thus $\bar{x}\bar{y}$ acts like a complement of $x + y$. So we can apply (10.8) to conclude that $\bar{x}\bar{y}$ is the complement of $x + y$. QED.

In the propositional calculus we can find a disjunctive normal form (DNF) and a conjunctive normal form (CNF) for any wff. These ideas carry over to any Boolean algebra, where $+$ corresponds to disjunction and $\cdot$ corresponds to conjunction. So we can make the following statement:

Any Boolean expression has a DNF and a CNF.

example 10.14 DNF and CNF Constructions

We'll construct a DNF and CNF for the expression

$$\overline{x + y} + z.$$

The following transformations do the job:

$$\begin{aligned}\overline{x + y} + z &= \overline{x} \overline{y} + z && \text{(DNF)} \\ &= (\overline{x} + z) (\overline{y} + z) && \text{(CNF)}.\end{aligned}$$

end example

10.2.2 Digital Circuits

Now let's see what Boolean algebra has to do with digital circuits. A *digital circuit* (also called a *logic circuit*) is an electronic representation of a function whose input values are either high or low voltages and whose output value is either a high or low voltage. Digital circuits are used to represent and process information in digital computers. The high-and low-voltage values are normally represented by the two digits 1 and 0.

The basic electronic components used to build digital circuits are called *gates*. The three basic “logic” gates are the AND gate, the OR gate, and the NOT gate. These gates work just like the corresponding logical operations, where 1 means true and 0 means false. So we can represent digital circuits as Boolean expressions with values in the Boolean algebra whose carrier is $\{0, 1\}$, where 0 means false, 1 means true, and the operations $+$, $\cdot$, and $\neg$ stand for $\vee$, $\wedge$, and $\neg$, respectively.

The three logic gates are represented graphically as shown in [Figure 10.4](#), where the inputs are on the left and the outputs are on the right.

Arithmetic Circuits

These gates can be combined in various ways to form digital circuits to do all basic arithmetic operations. For example, suppose we want to add two binary digits x and y . The first thing to notice is that the result has a summand digit and a carry digit. We'll consider two functions, “carry” and “summand.” Let's look at the carry function first. Notice that $\text{carry}(x, y) = 1$ if and only if $x = 1$ and $y = 1$. Thus we can define the carry function as follows:

$$\text{carry}(x, y) = xy.$$

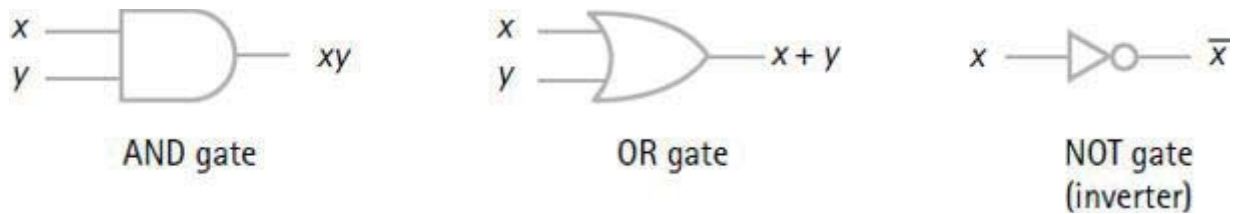


Figure 10.4 Logical gates.

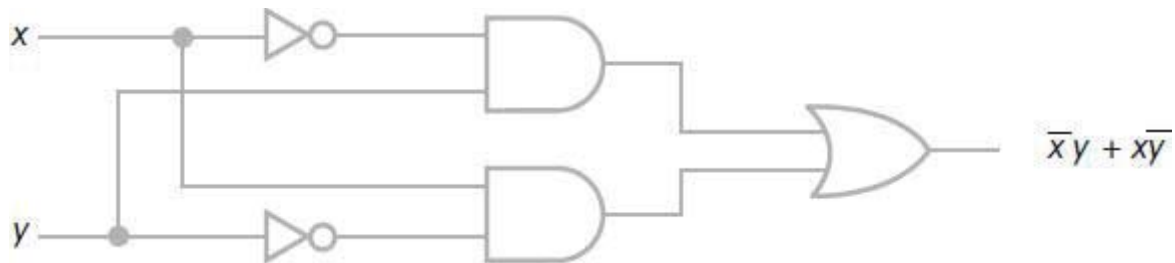


Figure 10.5 Summand circuit.

A circuit to implement the carry function consists of the simple AND gate shown in [Figure 10.4](#).

Now let's look at the summand. It's clear that $\text{summand}(x, y) = 1$ if and only if either $x = 0$ and $y = 1$ or $x = 1$ and $y = 0$. Thus we can define the summand function as follows:

$$\text{summand}(x, y) = \bar{x}y + x\bar{y}.$$

A circuit to implement the summand function is shown in [Figure 10.5](#).

We can combine the two circuits for the carry and the summand into one circuit that gives both outputs. The circuit is shown in [Figure 10.6](#). Such a circuit is called a *half-adder*, and it's a fundamental building block in all arithmetic circuits.

example 10.15 A Simple Half-Adder

Let's see whether we can simplify the circuit for a half-adder. The preceding circuit for a half-adder has six gates. But we can do better. First, notice that the expression for the summand, $\bar{x}y + x\bar{y}$ has five operations: two negations, two conjunctions, and one disjunction. Let's rewrite it as follows (be sure to fill in the reasons for each step).

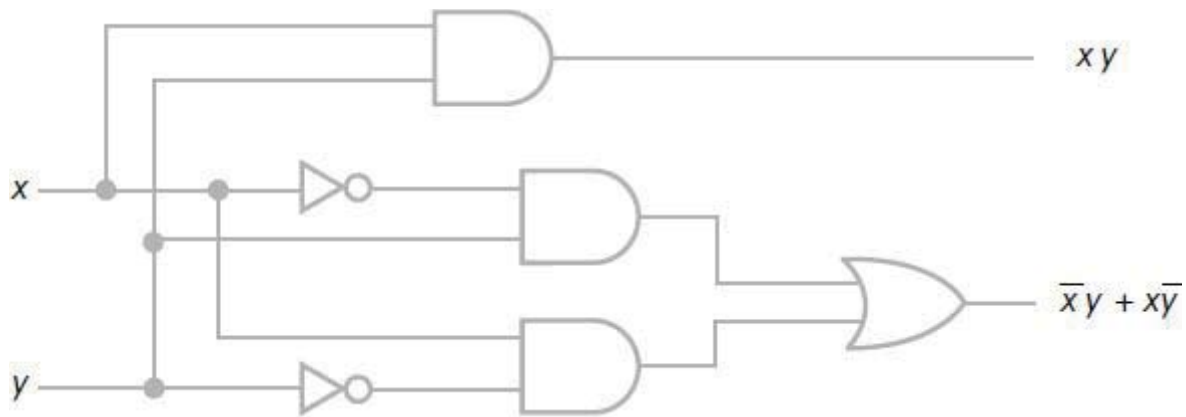


Figure 10.6 Half-adder circuit.

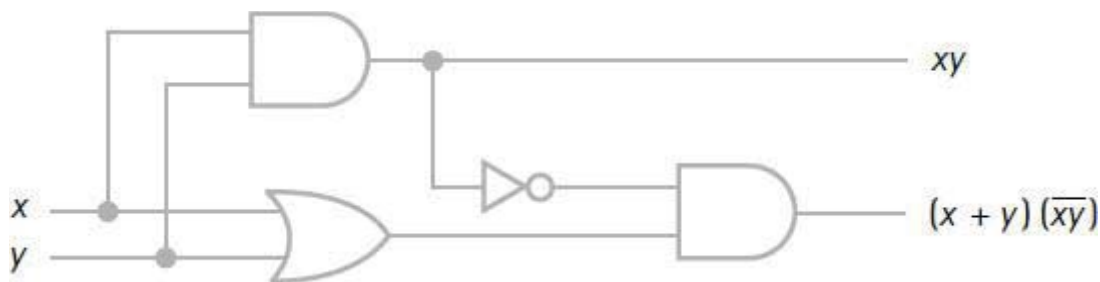


Figure 10.7 Simpler half-adder circuit.

$$\begin{aligned}
 \bar{x}y + x\bar{y} &= (\bar{x}y + x)(\bar{x}y + \bar{y}) \\
 &= (x + y)(\bar{x} + \bar{y}) \\
 &= (x + y)\overline{xy}.
 \end{aligned}$$

This latter expression has four operations: two conjunctions, one disjunction, and one negation. Also, note that the expression xy is computed before the negation is applied. Therefore, we can also use this expression for the carry. So we have a simpler version of the half-adder, as shown in [Figure 10.7](#).

end example

Constructing a Full Adder

We'll describe a circuit to add two binary numbers. For example, suppose we want to add the two binary numbers 1 0 1 1 and 1 1 1 0. The school method can be pictured as follows:

$$\begin{array}{rcccc}
 & 1 & 1 & 0 & \leftarrow \text{(carry bits)} \\
 & 1 & 0 & 1 & 1 \\
 & 1 & 1 & 1 & 0 \\
 \hline
 1 & 1 & 0 & 0 & 1
 \end{array}$$

So if we want to add two binary numbers, then we can start by using a half-adder on the two rightmost digits of each number. After that, we must be able to handle the addition of three binary digits: two binary digits and a carry from the preceding addition. A digital circuit to accomplish this latter feat is called a *full adder*.

We can build a full adder by using half-adders as components. Let's see how it goes. First, to get the big picture, we will denote a half-adder by a box with two input lines and two output lines, as shown in [Figure 10.8](#).

To get an idea about the kind of circuit we need, let's look at a table of values for the outputs sum and carry. [Figure 10.9](#) shows the values of sum and carry that are obtained by adding three binary digits.

Let's use [Figure 10.9](#) to find DNFs for the sum and carry functions in terms of x , y , and z . Notice that the value of the sum is 1 in four places, on lines 2, 3, 5, and 8 of the table. So the DNF for the sum function will consist of the disjunction of four terms, with each conjunction constructed from the values of x , y , and z on the four lines. Similarly, the value of the carry is 1 in four places, on lines 4, 6, 7, and 8. So the DNF for the carry function depends on conjunctions that depend on these latter four lines. We obtain the following forms for sum and carry.

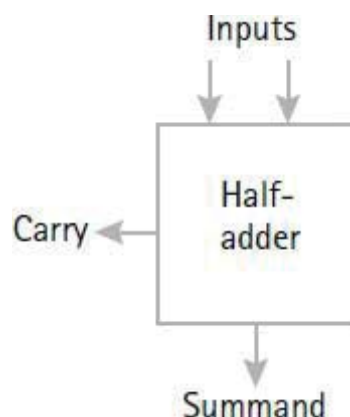


Figure 10.8 Half adder.

x	y	z	Sum	$Carry$
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 10.9 Adding three binary digits.

$$\begin{aligned}\text{sum}(x, y, z) &= \bar{x}\bar{y}z + \bar{x}y\bar{z} + x\bar{y}\bar{z} + xyz \\ &= \bar{x}(\bar{y}z + y\bar{z}) + x(\bar{y}\bar{z} + yz) .\end{aligned}$$

$$\begin{aligned}\text{carry}(x, y, z) &= \bar{x}yz + x\bar{y}z + xy\bar{z} + xyz \\ &= yz + x(\bar{y}z + y\bar{z}) .\end{aligned}$$

At this point we could build a circuit for sum and carry. But let's study the expressions that we obtained. Notice first that the expression

$$\bar{y}z + y\bar{z}$$

occurs in both the sum formula and the carry formula. Recall also that this is the expression for the summand output of the half-adder. It can be shown that the expression $\bar{y}z + y\bar{z}$, in the sum function, is equal to the negation of the expression $\bar{y}\bar{z} + yz$ (the proof is left as an exercise). In other words, if we let $e = \bar{y}z + y\bar{z}$, then we can write the sum in the following form:

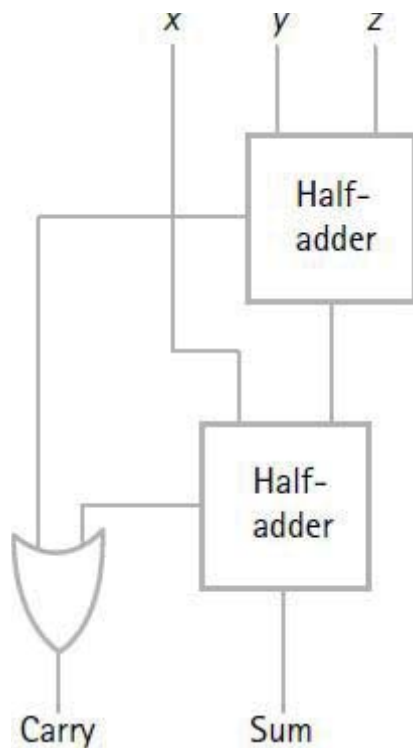


Figure 10.10 Full adder.

$$\text{sum}(x, y, z) = \bar{x}e + x\bar{e}.$$

This shows us that $\text{sum}(x, y, z)$ is just the summand output of a half-adder. So we can let y and z be inputs to a half-adder and then feed the summand output along with x into another half-adder to obtain the desired $\text{sum}(x, y, z)$. Before we draw the diagram, we need to look at the carry function. We have written carry in the following form:

$$\text{carry}(x, y, z) = yz + x(\bar{y}z + y\bar{z}).$$

Notice that the term yz is the carry output of a half-adder with input values y and z . Further, the term $x(\bar{y}z + y\bar{z})$ is the carry output of a half-adder with inputs x and $\bar{y}z + y\bar{z}$, where $\bar{y}z + y\bar{z}$ is the output of the half-adder with inputs y and z . So we can draw a picture of the circuit for a full adder as shown in [Figure 10.10](#).

Minimization

As we've seen in the previous two examples, we can get simpler digital circuits if we spend some time simplifying the corresponding Boolean expressions. Often a digital circuit must be built with the minimum number of components, where the components correspond to DNFs or CNFs.

This brings up the question of finding a *minimal* DNF for a Boolean expression. Here the word “minimal” is usually defined to mean the fewest number of fundamental conjunctions in a DNF, and if two DNFs have the same number of fundamental conjunctions, then the one with the fewest literals is minimal. The term *minimal* CNF is defined analogously. It's not always easy to find a minimal DNF for a Boolean expression.

example 10.16 A Minimal DNF

The Boolean expression $yz + yx$ is a minimal DNF for the expression

$$\bar{x}yz + xyz + xy\bar{z}.$$

We can show that these two expressions are equivalent as follows:

$$\begin{aligned}\bar{x}yz + xyz + xy\bar{z} &= (\bar{x} + x)yz + xy\bar{z} \\ &= yz + xy\bar{z} \\ &= y(z + x\bar{z}) \\ &= y(z + x) \\ &= yz + yx.\end{aligned}$$

But it takes some work to see that $yz + xy$ is a minimal DNF. First we need to argue that there is no equivalent DNF with just a single fundamental conjunction. Then we need to argue that there is no equivalent DNF with two fundamental conjunctions with fewer than four literals.

end example

There are formal methods that can be applied to the problem of finding minimal DNFs and minimal CNFs. We'll leave them to more specialized texts.

Exercises

The Axioms

1. Let S be a set and $B = \text{power}(S)$. Suppose someone claims that B is a Boolean algebra with the following definitions for the operators.

$+$ is union,
 $\cdot$ is difference,
 $-$ is complement with respect to S ,
 0 is S ,
 1 is $\emptyset$.

Is the result a Boolean algebra? Why or why not?

Boolean Expressions

2. Prove each of the following four absorption laws (10.7).

a. $x + xy = x$.

b. $x(x + y) = x$.

c. $x + \overline{x}y = x + y$.

d. $x(\overline{x} + y) = xy$.

3. Let $e = \overline{y}z + y\overline{z}$. Prove that $\overline{e} = \overline{y}\overline{z} + yz$.

4. Use Boolean algebra properties to prove each of the following equalities.

a. $\overline{x} + \overline{y} + xyz = \overline{x} + \overline{y} + z$.

b. $\overline{x} + \overline{y} + xy\overline{z} = \overline{x} + \overline{y} + \overline{xy}z$.

5. Simplify each of the following Boolean expressions.

a. $x + \overline{x}y$.

b. $x\overline{y}\overline{x} + xy\overline{x}$.

c. $\overline{x}\overline{y}z + x\overline{y}\overline{z} + x\overline{y}z$.

d. $xy + x\overline{y} + \overline{x}y$.

e. $x(y + \overline{y}z) + \overline{y}z + yz$.

f. $x(y + z)(\overline{x} + y)(\overline{y} + x + z)$.

g. $x + yz + \overline{x}y + \overline{y}xz$.

h. $x + y(x + y)$.

i. $(x + y) + \overline{x}\overline{y}$.

j. $(x + y)(\overline{y} + x)(\overline{x} + y)$.

k. $xy + x + y$.

l. $(x + y)xy$.

6. For each part of Exercise 2, draw two logic circuits. One circuit should implement the expression on the left side of the equality. The other circuit should implement the expression on the right side of the equality. Each circuit should use the same number of gates as there are operations in the expression.

7. Write down the dual of each of the following Boolean expressions.

a. $x + 1$.

b. $x(y + z)$.

c. $xy + xz$.

d. $xy + z$.

e. $y + \overline{x}z$.

f. $z\overline{y}\overline{x} + xz\overline{x}$.

8. Show that, in a Boolean algebra, $1 + x = 1$ for every element x .

9. Show that, in a Boolean algebra, $\overline{xy} = \overline{x} + \overline{y}$ for all elements x and y .

Challenges

10. Let B be the carrier of a Boolean algebra. Suppose B is a finite set, and suppose $0 \neq 1$. Show that the cardinality of B is an even number.

11. A Boolean algebra can be made into a partially ordered set by letting $x \preceq y$ mean $x = xy$.

a. Show that $\preceq$ is reflexive, antisymmetric, and transitive.

b. Show that $x \preceq y$ if and only if $y = x + y$.

12. A Boolean algebra, when considered as a poset—as in Exercise 11—is

also a lattice. Prove that $\text{glb}(x, y) = xy$ and $\text{lub}(x, y) = x + y$.

13. In Example 10.13 we considered the set B_n of positive divisors of n together with the operations of lcm, gcd, n/x , where n is one and 1 is zero. Prove that this algebra is not Boolean if a prime p occurs more than once as a factor of n . *Hint*: Consider the complement of p .

10.3 Abstract Data Types As Algebras

Programming problems involve data objects that are processed by a computer. To process data objects, we need operations to act on them. So algebra enters the programming picture. In computer science, an *abstract data type* consists of one or more sets of data objects together with one or more operations on the sets and some axioms to describe the operations. In other words, an abstract data type is an algebra. There is, however, a restriction on the carriers of abstract data types. A carrier must be able to be constructed in some way that will allow the data objects and the operations to be implemented on a computer.

Programming languages normally contain some built-in abstract data types. But it's not possible for a programming language to contain all possible ways to represent and operate on data objects. Therefore, programmers must often design and implement new abstract data types. The axioms of an abstract data type can be used by a programmer to check whether an implementation is correct. In other words, the implemented operations can be checked to see whether they satisfy the axioms.

An abstract data type allows us to program with its data objects and operations without having to worry about implementation details. For example, suppose we need to create an abstract data type for processing polynomials. We might agree to use the expression $\text{add}(p, q)$ to represent the sum of two polynomials p and q . To implement the abstract data type, we might represent a polynomial as an array of coefficients and then implement the add operation by adding corresponding array components. Of course, there are other interesting and useful ways to represent polynomials and their addition. But no matter what implementation is used, the statement $\text{add}(p, q)$ always means the same thing. So we've abstracted away the implementation details.